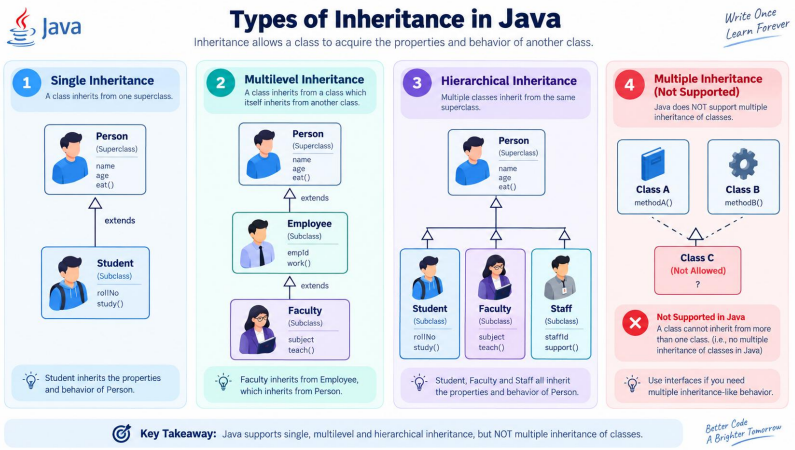
CSC241 — Object-Oriented Programming

Lecture 14 — Inheritance in Java

Single, multilevel, and hierarchical inheritance • extends • super



Dr Muhammad Shahid Bhatti
Assistant Professor
Department of Computer Science
Office H-17
GitHub: github.com/mshahidbhatti/CSC241-OOP

Inheritance motivation | extends | Single | Multilevel | Hierarchical | super | Live coding

Reference: CSC241 Lecture Plan • Liang, Ch. 11

Bridge from Lecture 13

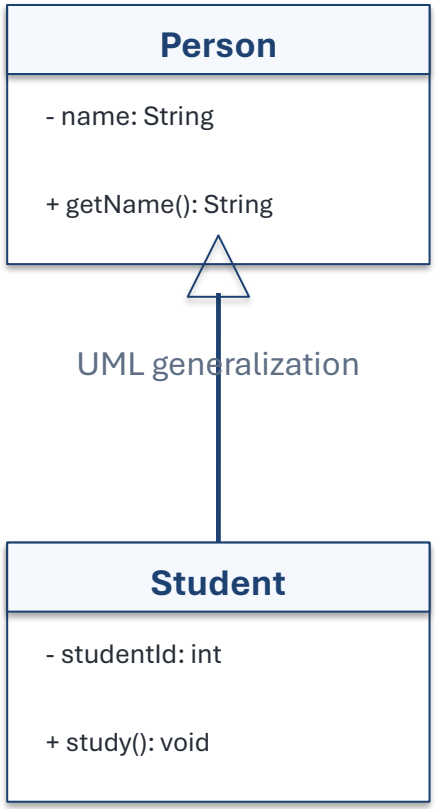
Last lecture designed hierarchies conceptually. Today we implement them in Java.

Lecture 13 asked: “Should these classes form a hierarchy at all?”

Lecture 14 asks: “How do we express a valid hierarchy in Java?”

We move from generalization in UML to extends in code.

We also introduce super for referring to the superclass and its constructor/member behavior.



Java mapping

```
public class Student extends Person {
    // Student-specific members
}
```

Learning outcomes

By the end of this lecture, students should be able to:

L14

explain inheritance using superclass/subclass and IS-A terminology;

implement single, multilevel, and hierarchical inheritance using extends;

identify inherited members versus subclass-specific members;

explain why constructors are not inherited and why private members are not directly accessible;

use super(...) to select a superclass constructor and recognize the role of super for superclass members;

compile and debug a small multi-file inheritance hierarchy from Command Prompt.

Live-coding rhythm

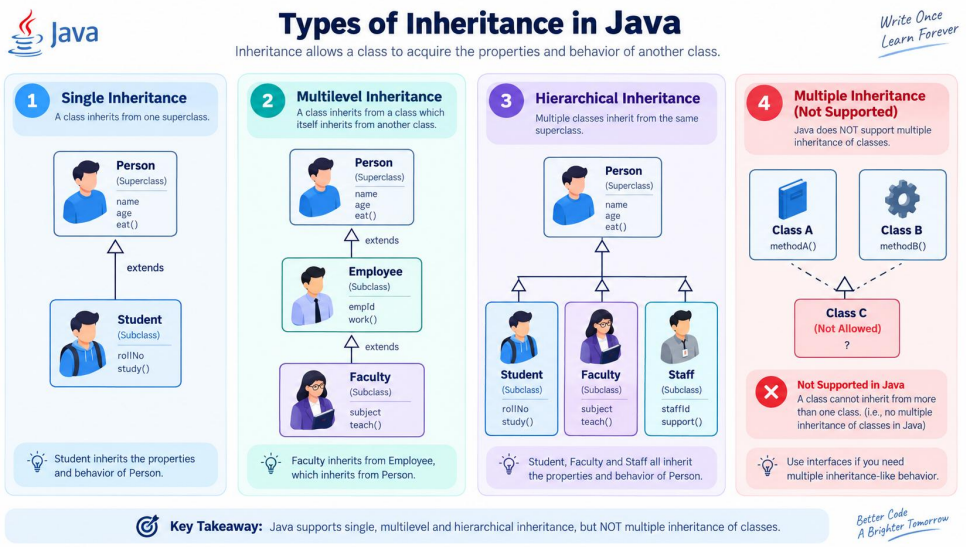
Predict → write one class → compile → add subclass → compile → instantiate → trace state → break code intentionally → fix.

Scope boundary

Method overriding, Object.toString(), and detailed protected behavior are treated more deeply in Lecture 16. Constructor-chain rules are continued in Lecture 15.

Lecture roadmap

1. Why inheritance exists
2. Superclass, subclass, and IS-A
3. extends and inherited members
4. Single / multilevel / hierarchical inheritance
5. super keyword and superclass construction
6. Access-control implications
7. Live coding, tracing, and debugging



Core question

When does a subclass genuinely represent a more specific kind of its superclass?

Motivation — repeated state and behavior

Without a meaningful hierarchy, related classes often duplicate code.

Before inheritance

```
class Student {
    String name;
    String email;
    void printContact() { ... }
    int studentId;
}

class Faculty {
    String name;
    String email;
    void printContact() { ... }
    String designation;
}
```

name, email, and printContact() describe a common concept.

Duplicated logic can drift when requirements change.

A shared superclass can centralize truly common state and behavior.

But inheritance is justified only when the relationship is a real IS-A relationship.

Design first

Do not create a superclass merely because two classes share one field. Search for a stable common abstraction.

14

Part I — Core Inheritance Concepts

Superclass, subclass, extends, and inherited behavior

Inheritance mental model

A subclass is a more specific class that builds on a superclass.

L14



Read it aloud

“Student IS-A Person.” Therefore a Student can reuse the accessible behavior of Person and add Student-specific state/behavior.

Key terminology

Superclass / base class	The more general class being extended.
Subclass / derived class	The more specific class that extends a superclass.
Inheritance hierarchy	A tree-like organization of classes connected by IS-A relationships.
Inherited member	An accessible field/method that becomes available through the superclass relationship.
Specialization	Adding or refining behavior/state in the subclass.

The extends keyword

Java class inheritance uses a single explicit superclass.

Person.java

```
public class Person {
    private String name;

    public Person(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }
}
```

Student.java

```
public class Student extends Person {
    private int studentId;

    public Student(String name, int studentId) {
        super(name);
        this.studentId = studentId;
    }

    public void study() {
        System.out.println(getName() + " is studying");
    }
}
```

Observe

Student declares only student-specific data/behavior. It can call the accessible getName() behavior inherited from Person.

What a subclass gets — and what it adds

Accessible instance methods from its superclass.

Accessible fields, depending on visibility and package context.

A superclass portion of object state created during construction.

The ability to add new fields, constructors, and methods.

Accessible inherited instance methods (subject to Java access rules).

Superclass fields that are visible in the subclass context.

One Student object holds its Person state plus Student-specific state.

Subclass declarations can add new fields, constructors, and methods.

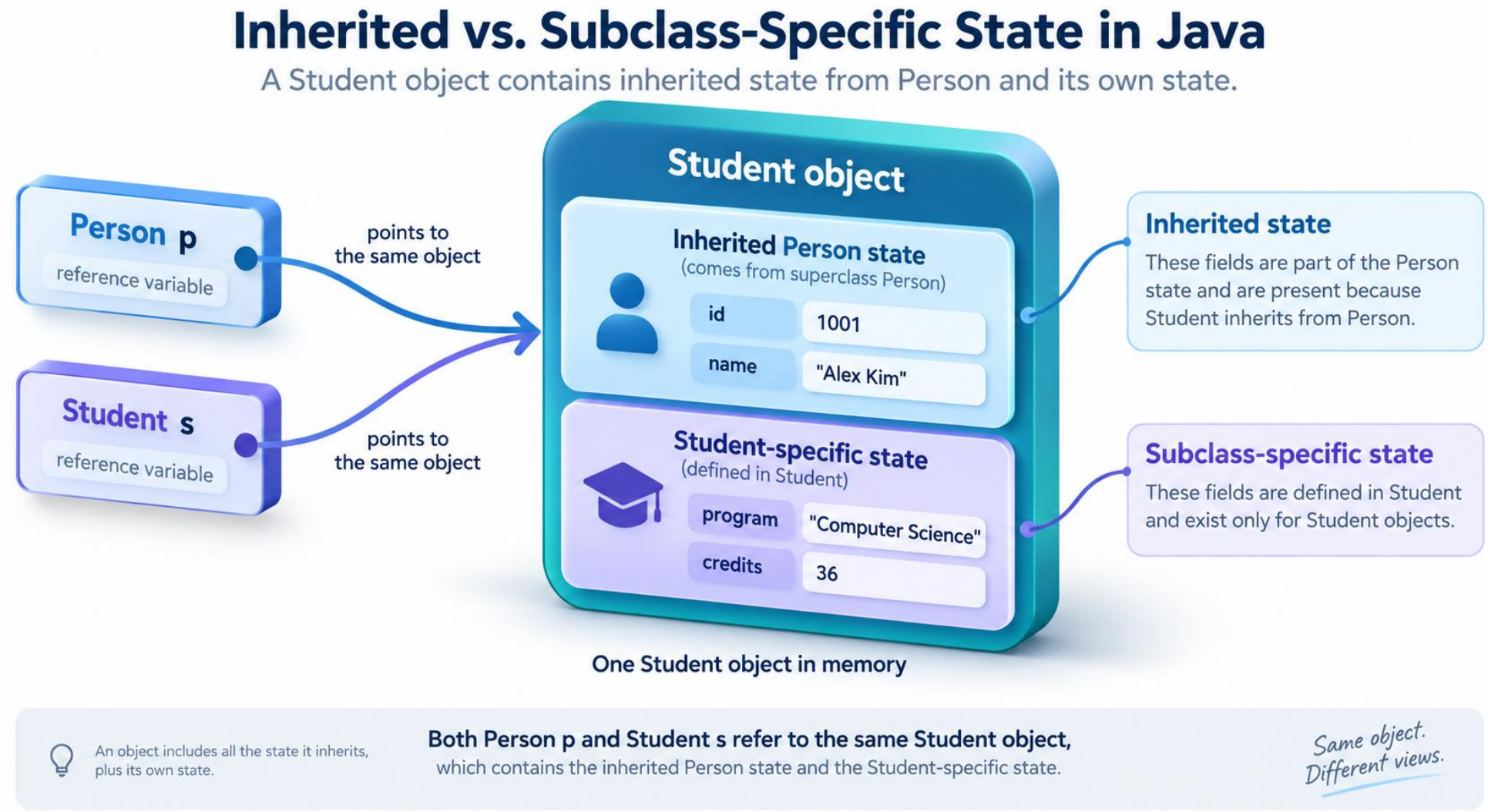
Important distinction

A subclass object contains superclass state, but private superclass fields are not directly accessible by subclass source code. Use inherited public/protected behaviour to work with that state.

Example

Student can call getName() inherited from Person even though Person.name itself is private.

One subclass object contains inherited and subclass-specific state



Calling inherited methods

The subclass object can use inherited accessible methods as if they belong to the subclass interface.

Demo.java

```
public class Demo {
    public static void main(String[] args) {
        Student s = new Student("Ali", 101);

        System.out.println(s.getName()); // inherited
        s.study();                       // Student-specific
    }
}
```

getName() is defined in Person.

study() is defined in Student.

The object presents both inherited and subclass-specific behavior.

This unified object model is one of the core benefits of inheritance.

Classroom question

Which class contains the source code for getName()? Which object executes it when s.getName() is called?

What is not inherited directly?

Constructors and private members deserve special attention.

Constructors are not inherited by subclasses.

Private members of the superclass are not inherited as accessible members of the subclass.

A subclass object still contains the superclass portion/state established by the superclass constructor.

Static members belong to classes and follow different access/lookup rules; avoid treating them as instance inheritance in your mental model.

Private superclass state

```
class Person {
    private String name;
    public String getName() {
        return name;
    }
}

class Student extends Person {
    void test() {
        // name = "X"; // ERROR: name is private
        System.out.println(getName()); // OK
    }
}
```

initialize the corresponding class portion.

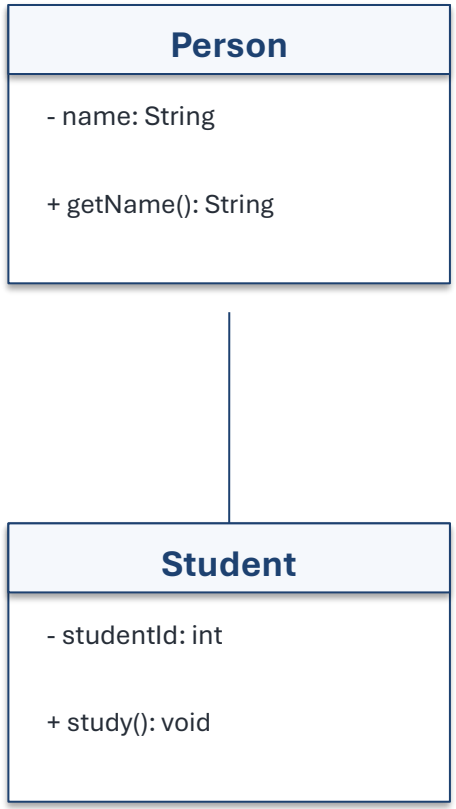
Private fields exist in the object but cannot be named directly in subclass source.

Static members belong to their declaring class; avoid confusing class-level access with instance state.

Inherited public methods can provide controlled access to private superclass state.

Single inheritance

One subclass directly extends one superclass.



Java syntax

```
public class Student extends Person {
    // one direct superclass: Person
}
```

Simple and common.

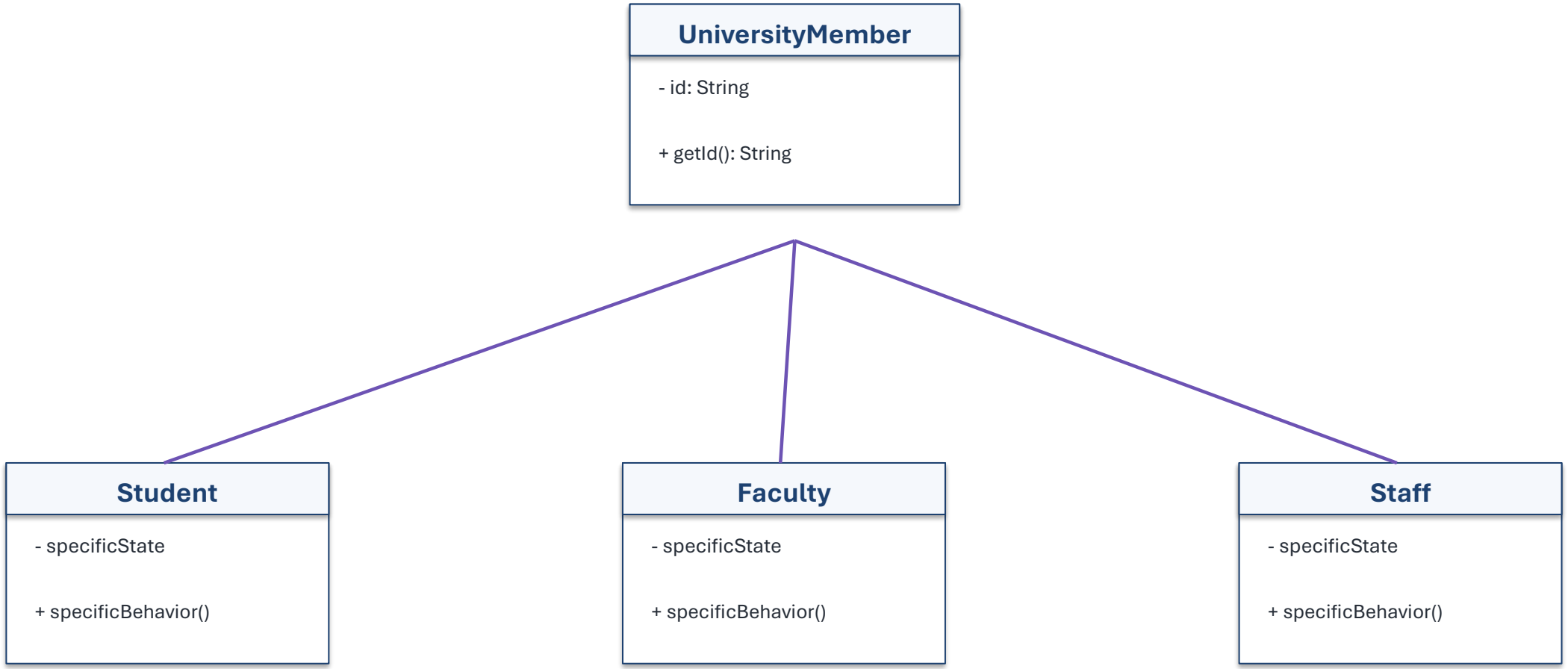
Student inherits from Person.

Student may introduce its own state/behavior.

A Java class can directly extend only one class.

Hierarchical inheritance

Several subclasses share the same superclass.



Why useful

Common UniversityMember data/behavior lives once. Student, Faculty, and Staff specialize it independently.

Multilevel inheritance

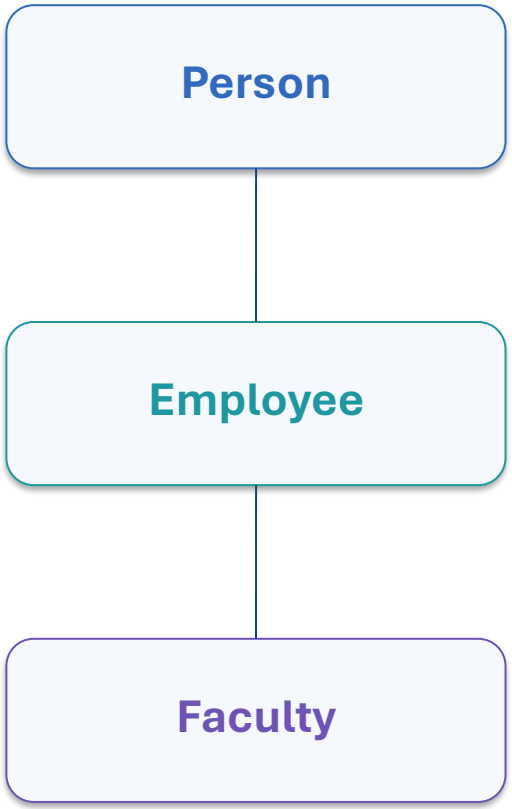
A subclass can itself become the superclass of another class.

Three-level hierarchy

```
class Person { ... }

class Employee extends Person {
    private String employeeId;
}

class Faculty extends Employee {
    private String department;
}
```



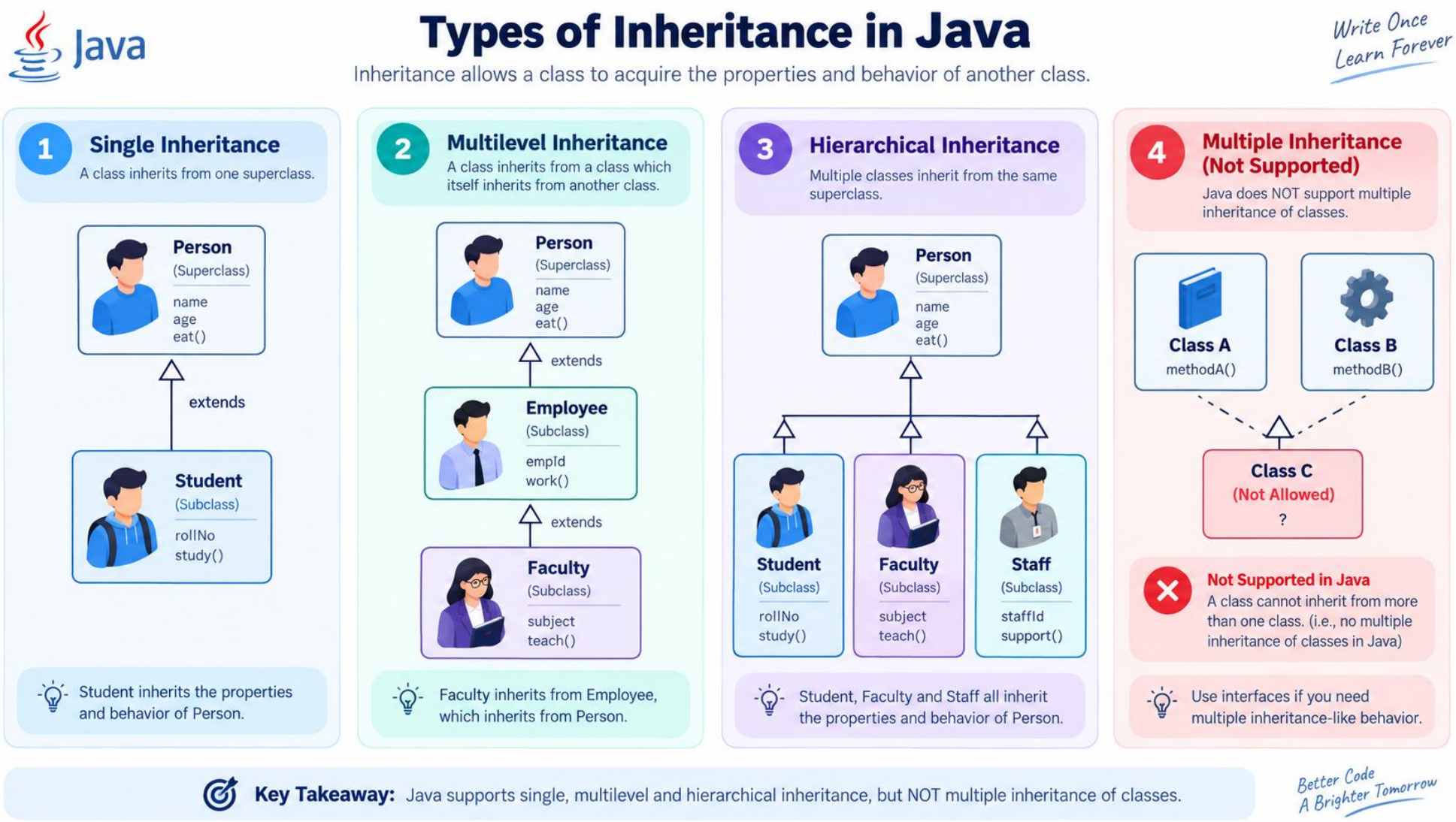
Faculty is-an Employee.

Employee is-a Person.

Therefore Faculty also has the accessible behavior inherited through Employee from Person.

Deep hierarchies should remain shallow enough to reason about.

Types of inheritance supported by Java classes



Why a class cannot extend two classes

Java avoids direct multiple inheritance of classes.


```
// Invalid Java: a class may extend only one class
class TeachingAssistant extends Student, Employee {
}
```

```
// Not legal Java
class TeachingAssistant extends Student, Employee {
    // ...
}
```

A class may directly extend only one superclass.

This avoids ambiguity when multiple parents contribute conflicting state or behavior.

Later, interfaces provide multiple behavioral contracts without multiple class inheritance.

Do not confuse “implements multiple interfaces” with “extends multiple classes.”

Course bridge

Multiple inheritance through interfaces is handled later when interfaces are introduced formally.

The IS-A test before you write extends

Inheritance must express genuine IS-A specialization; the full behavioral-substitution principle will be developed during polymorphism.

✓ **Student IS-A Person**

✗ **Car IS-A Engine**

✓ **Faculty IS-A UniversityMember**

✗ **CourseSection IS-A Room**

✓ **Car IS-A Vehicle**

✗ **Student IS-A Enrollment**

Rule of thumb

If the sentence “X is a Y” sounds conceptually wrong, prefer composition/association instead of inheritance.

Inheritance for meaning, not just code reuse

Good use

Student extends UniversityMember

Car extends Vehicle

SavingsAccount extends BankAccount

Suspicious use

Car extends Engine

Report extends Printer

CourseSection extends Room

Why?

The right-side examples confuse HAS-A or USES-A with IS-A.

14

Part II — The super Keyword

Superclass constructors and superclass-member access

What does super mean?

Inside a subclass, super refers to the superclass context.

super(...) invokes a superclass constructor.

super.member can refer to an accessible superclass field/member.

super.method(...) can explicitly invoke superclass behavior.

super is not an object variable you can reassign or store.

Constructor-call rules are explored in greater depth in Lecture 15.

Two uses of super

```
class Student extends Person {
  private int id;
  Student(String name, int id) {
    super(name);
    this.id = id;
  }
  void printInfo() {
    System.out.println(super.getName());
  }
}
```

Subclass construction and super(...)

Java Subclass Construction and the super Keyword

When you create a subclass object, the superclass part is constructed first. The `super` keyword lets the subclass call the superclass constructor and access superclass members.

Build on what's already there — that's the power of super.

Superclass: Person

```
class Person {
  private String name;

  public Person(String name) {
    this.name = name;
  }

  public String getName() {
    return name;
  }
}
```

Person state

name	"Alice"

Subclass: Student

```
class Student extends Person {
  private int studentId;

  public Student(String name, int studentId) {
    super(name);   // calls Person constructor
    this.studentId = studentId;
  }

  public int getStudentId() {
    return studentId;
  }
}
```

Student state

name	"Alice"
studentId	1001

Object Construction Sequence

`new Student("Alice", 1001)`

1 Allocate Student object

Student object (in memory)

name	(not set yet)
studentId	(not set yet)

Memory is allocated for a Student object. Fields have default values (null, 0, etc.).

2 Call super(name)

```
super(name);
```

The first line of the Student constructor calls the Person constructor with the given name.

3 Initialize Person state

Person part (inside Student)

name	"Alice"

The Person constructor runs and initializes the inherited state (name).

4 Return to Student constructor

Control returns to the Student constructor after the Person constructor completes.

5 Initialize Student-specific state

Student object (ready)

name	"Alice"
studentId	1001

The Student constructor initializes its own fields. The object is now fully constructed and ready to use.

Result: a Student object whose Person part is correctly initialized.

Superclass first, then subclass!

Using super(name) in the constructor — Java

```
class Student extends Person {
  private int studentId;

  public Student(String name, int studentId) {
    super(name);   // calls Person(String name)
    this.studentId = studentId;
  }
}
```

Using super.getName() to access a superclass member — Java

```
class Student extends Person {
  // ...
  public void printInfo() {
    System.out.println("Name: " + super.getName());
    System.out.println("Student ID: " + studentId);
  }
}
```

Key Takeaways

- A subclass constructor must call a superclass constructor (either explicitly with `super(...)` or implicitly).
- `super(name)` calls the Person constructor to initialize inherited state.
- You can use `super` to access superclass methods and fields, e.g., `super.getName()`.
- The superclass part is constructed before the subclass part.

Implicit super() — and a common compile error

When a constructor has no explicit invocation, the compiler inserts a call to the no-argument superclass constructor (subject to Java constructor rules).

Problem

```
class Person {
    Person(String name) { }
}
class Student extends Person {
    private int id;
    Student(int id) {
        this.id = id; // ERROR: implicit
super()
    }
}
```

Fix

```
class Student extends Person {
    private int id;
    Student(String name, int id) {
        super(name); // existing
constructor
        this.id = id;
    }
}
```

Person has no no-argument constructor.

Therefore implicit super() cannot compile.

Student must explicitly call an existing Person constructor.

Using super to access superclass members

super.method() explicitly chooses an accessible superclass implementation; ordinary inherited calls often do not need super.

Explicit superclass method call

```
class Person {
    public String getLabel() {
        return "Person";
    }
}

class Student extends Person {
    public String getStudentLabel() {
        return super.getLabel() + " -> Student";
    }
}
```

super.getLabel() explicitly starts lookup from Person.

This can clarify which superclass implementation/member you intend to use.

Detailed overriding behavior comes later; for now, focus on how super reaches the superclass layer.

this vs. super

Keyword	Meaning inside an instance context	Typical use
this	the current object / current class context	this.field, this(...), current object behavior
super	the superclass context of the current object	super(...), super.method(), superclass member access

Constructor reminder

A constructor begins with either this(...) or super(...), not both as separate first statements. Full constructor-chain rules continue next lecture.

Access control across an inheritance hierarchy

Inheritance does not erase encapsulation.

public	Accessible from subclass and clients (subject to object/reference context).
protected	Accessible in package and in subclasses under Java's protected rules.
package/default	Accessible only when subclass is in the same package.
private	Not directly accessible in subclass source code.

Good design

Prefer private superclass state with meaningful protected/public behavior rather than exposing fields merely to make inheritance convenient.

Superclass private state still belongs to the object

The subclass should use superclass behavior rather than bypassing encapsulation.

Person.java

```
class Person {
    private String name;

    public Person(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }
}
```

Student.java

```
class Student extends Person {
    Student(String name) {
        super(name);
    }

    void introduce() {
        // System.out.println(name);  // not accessible
        System.out.println(getName()); // correct
    }
}
```

Mental model

The name state is part of the Person portion of the Student object. The superclass remains responsible for controlling access to it.

Preview — building a subclass object

The superclass part must be initialized before subclass-specific state.

1

Allocate memory

Space for the complete Student object.

2

Superclass construction

Person constructor initializes Person state.

3

Subclass construction

Student constructor initializes Student-specific state.

4

Ready object

The fully initialized Student object can now be used.

Next lecture

Lecture 15 formalizes constructor chaining, explicit superclass-constructor selection, and chaining order through hierarchies.

14

Part III — Live Coding & Reasoning

Build, compile, trace, break, and repair inheritance code

Live coding checkpoint 1 — Person → Student

Type from scratch rather than pasting the final code.

Create Person.java with private name and getName().

Compile Person.java.

Create Student.java extends Person.

Add studentId and a constructor using super(name).

Compile both classes.

Create Demo.java, instantiate Student, and call inherited + subclass-specific methods.

Command Prompt

```
javac Person.java
javac Student.java
javac Demo.java
java Demo
```

Teaching move

Before each compile, ask students what new .class files or errors they expect. Let the compiler validate their mental model.

Live coding checkpoint 2 — hierarchical inheritance

One superclass, three specialized subclasses.

L14

UniversityMember.java

```
public class UniversityMember {
    private String id;
    private String name;

    public UniversityMember(String id, String
        this.id = id;
        this.name = name;
    }

    public String getName() { return name; }
}
```

Student.java

```
public class Student extends
UniversityMember {
    private String program;
    public Student(String id,
String name, String program) {
        super(id, name);
        this.program = program;
    }
    public void study() {

System.out.println(getName() + "
studies");
    }
}
```

Faculty.java

```
public class Faculty extends UniversityMember
    private String designation;
    public Faculty(String id, String name, Str
        super(id, name);
        this.designation = designation;
    }
}
```

Extend the exercise

Create this hierarchy in a SEPARATE folder from Checkpoint 1 (both examples define Student.java). Add Staff as the third subclass. Identify shared versus specialized state.

Compiling a multi-file hierarchy from Command Prompt

Suggested workflow

Create a fresh folder for this example before compiling, to avoid mixing two unrelated Student classes.

```
cd Lecture14_Hierarchy
javac UniversityMember.java Student.java Faculty.java Staff.java Demo.java
java Demo
```

Use dir to inspect generated .class files; rebuild after changing the hierarchy.

javac may compile required source dependencies automatically when source files are available.

For teaching, compiling step-by-step makes class dependencies visible.

Use dir to show generated .class files.

Delete .class files occasionally and rebuild to demonstrate source-to-bytecode flow.

Trace the code before running it

Prediction exercise

```
Student s = new Student("U001", "Sara", "BCS");

System.out.println(s.getName());
s.study();

UniversityMember u = s;    // preview only
System.out.println(u.getName());
```

How many objects are created?

Which constructor(s) must execute?

Which class defines getName()?

Which class defines study()?

What state exists inside the Student object?

Preview only

The line UniversityMember u = s; introduces a supertype reference to a subtype object. We will study polymorphism formally later.

Compile–debug–run challenge

Find the problems before asking javac.

Broken on purpose

```
class Person {
    private String name;
    Person(String name) { this.name = name;
}
}
class Student extends Person {
    private int id;
    Student(int id) {
        this.id = id; // implicit super()
fails
    }
    void printName() {
        System.out.println(name); // private
access error
    }
}
```

1. Why does Student(int) fail without super(name)?
2. Why is Person.name inaccessible directly?
3. Which public/protected method should Person provide?
4. Write the corrected Student(String, int).
5. Predict compiler feedback BEFORE running javac.

Expected thinking

Fix design and syntax one issue at a time, then recompile. Inheritance errors become much easier when students reason in layers: superclass first, subclass second.

Common misconceptions to eliminate early

“extends means copy-and-paste the superclass code.” → No: it defines a type relationship and inherited member behavior.

“Private superclass fields disappear in subclass objects.” → No: superclass state exists, but direct access is restricted.

“Constructors are inherited.” → No: subclass constructors call superclass constructors.

“Java classes can extend two classes.” → No: one direct superclass only.

“Inheritance is always better than composition.” → No: use inheritance only for genuine IS-A relationships.

“super is another object.” → No: it is a keyword selecting the superclass context.

Inheritance design checklist

Ask these questions before writing extends:

- Is the subclass truly a more specific kind of the superclass?
- Are the common fields/behaviors stable and meaningful?
- Would clients reasonably treat the subclass as a superclass concept?
- Am I inheriting for meaning rather than only to reuse a few lines?
- Would composition model the relationship more naturally?
- Is the hierarchy shallow and understandable?
- Are superclass invariants and encapsulation preserved?

Professional habit

Good inheritance starts with domain reasoning, not with the extends keyword.

Link to Lecture 13

Hierarchy design decisions from the previous lecture should drive the code structure you implement today.

In-class design + coding activity (15–20 min)

Choose the correct relationship, then implement one hierarchy.

L14

A. UniversityMember → Student / Faculty / Staff

B. Vehicle → Car / Bus / Motorcycle

C. BankAccount → SavingsAccount / CurrentAccount

D. CourseSection → Room? Decide whether inheritance is correct before coding.

Deliverable

1) small hierarchy sketch, 2) Java class files, 3) one super(...) constructor call, 4) compile/run output, 5) one sentence defending the IS-A relationship.

Pair discussion

For option D, students should conclude that CourseSection HAS-A/USES-A Room rather than IS-A Room.

Constructor execution: predict before running

A Student object contains one Person subobject, not a separate Person instance.

```
class Person {
  Person(String name) {
    System.out.println("Person: " + name);
  }
}
class Student extends Person {
  Student(String name) {
    super(name);
    System.out.println("Student");
  }
}
```

new Student("Sara");

Output:
Person: Sara
Student

Question: Is Person a second object?
Answer: No. The Person portion is initialized as part of the same Student object.

Diagnose inherited access: three cases

Access depends on where the declaration appears and which visibility modifier it has.

```
class Person {
  private String name;
  public String getName() {
    return name;
  }
}
class Student extends Person {
  void show() {
    System.out.println(getName());
  }
}
```

✓ getName() is inherited and callable.

✕ Direct use of name fails (private).

✓ Person state is still inside the Student object.

Discussion: How would behavior change if name were protected? Why is private still preferable?

Lecture recap & exit ticket

Inheritance converts a valid conceptual hierarchy into reusable Java type structure.

L14

extends establishes a direct superclass-subclass relationship.

Java supports single, multilevel, and hierarchical class inheritance.

A subclass object combines superclass state/behavior with subclass-specific state/behavior.

Constructors are not inherited; superclass construction still occurs.

super selects superclass constructor/member context.

Inheritance should express IS-A — not merely code reuse.

Exit ticket

1) Write one valid extends declaration.
2) Why can Student not directly access a private Person field?
3) What does super(name) do?
4) Give one hierarchical-inheritance example.
5) Give one relationship that should use composition instead.

Next lecture

Lecture 15 — Constructor Chaining in Inheritance Hierarchies: chaining order, superclass constructor selection, inherited vs. non-inherited private state, and explicit super(...).